



Σχεδίαση και Υλοποίηση Αλγορίθμων Επεξεργασίας Εικόνας σε Διατάξεις FPGA

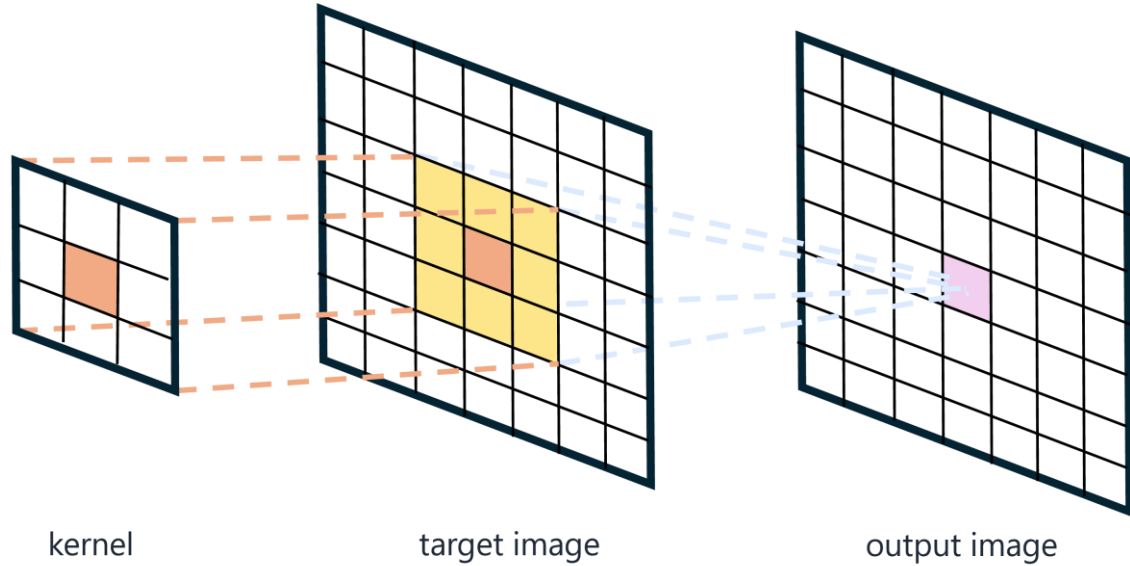
Νεφέλη Ζήκου

AEM: 15402

Επιβλέπων Καθηγητής: Κωνσταντίνος Σιώζιος

Spatial filtering

Calculation of target-pixel filtered value using its neighbors



Convolution of the
kernel with the image

$$\begin{bmatrix} P_{00} & P_{01} & P_{02} \\ P_{10} & P_{11} & P_{12} \\ P_{20} & P_{21} & P_{22} \end{bmatrix} * \begin{bmatrix} K_{00} & K_{01} & K_{02} \\ K_{10} & K_{11} & K_{12} \\ K_{20} & K_{21} & K_{22} \end{bmatrix} =$$

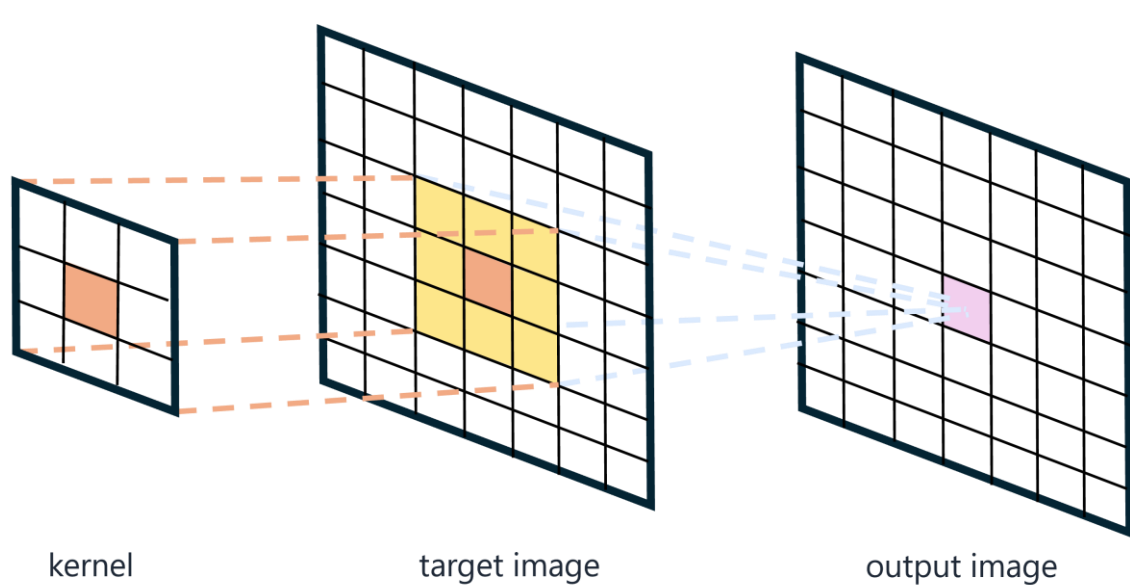
multiply-accumulate of kernel
coefficients & pixel neighbors

$$= P_{00} \cdot K_{00} + P_{01} \cdot K_{01} + P_{02} \cdot K_{02} \\ + \dots + P_{22} \cdot K_{22}$$

Filtered value of center pixel

Spatial filtering

Calculation of target-pixel filtered value using its neighbors



Convolution of the
kernel with the image

$$\begin{bmatrix} P_{00} & P_{01} & P_{02} \\ P_{10} & P_{11} & P_{12} \\ P_{20} & P_{21} & P_{22} \end{bmatrix} * \begin{bmatrix} K_{00} & K_{01} & K_{02} \\ K_{10} & K_{11} & K_{12} \\ K_{20} & K_{21} & K_{22} \end{bmatrix} =$$

multiply-accumulate of kernel
coefficients & pixel neighbors

$$= P_{00} \cdot K_{00} + P_{01} \cdot K_{01} + P_{02} \cdot K_{02} \\ + \dots + P_{22} \cdot K_{22}$$



Filtered value of center pixel

HD video

~100million pixels per second

+

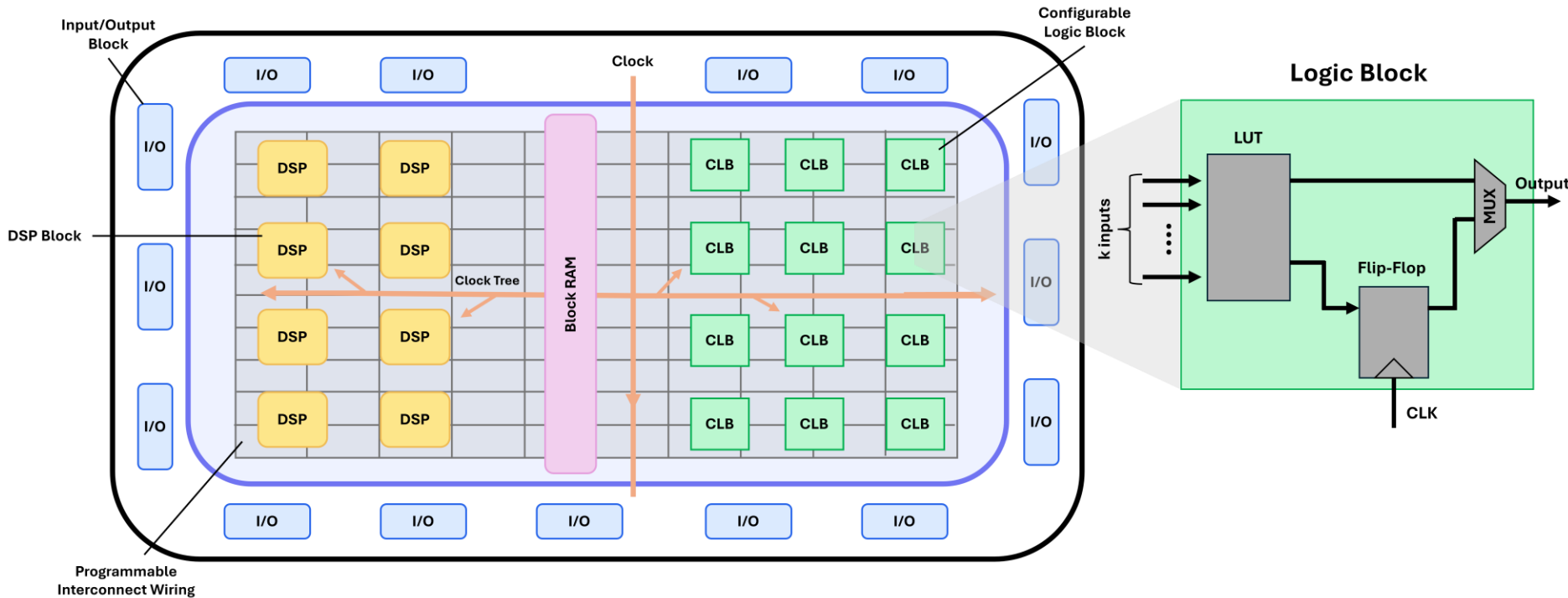
Convolution with a 3x3 matrix

9 MAC operations



**1 billion MAC
operations per second**

FPGA



Basic elements:

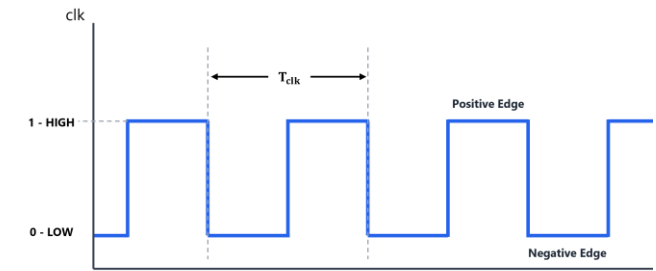
- Logic Blocks
- DSP blocks
- On-chip memory
- Input/Output interfaces
- Clock Infrastructure
- Programmable Interconnect

Digital Signal Processing Blocks (DSPs)

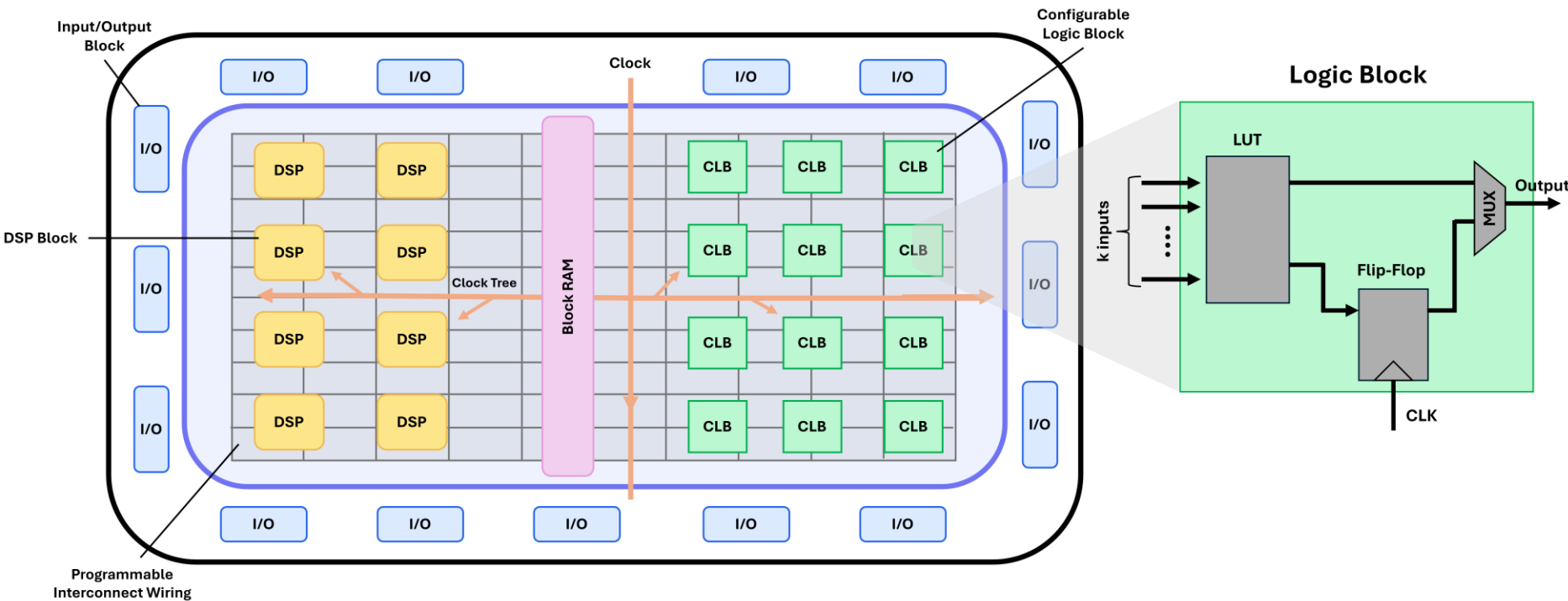
arithmetic unit
designed for heavy computations
supports MAC operations

Clock

Synchronizes elements
State transitions on clock edge
(positive or negative)



FPGA



Flip-Flops

Storage elements
data registering from input
to output
measure of sequential cost

Look-Up Tables

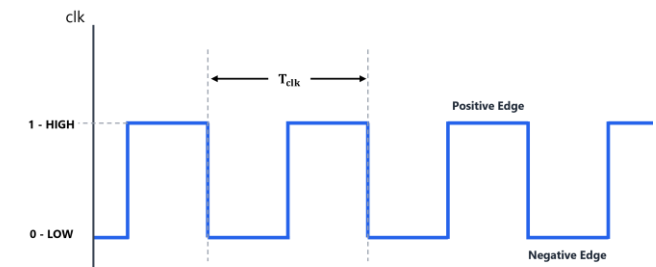
execute combinational logic
measure of logic cost

Digital Signal Processing Blocks (DSPs)

arithmetic unit
designed for heavy computations
supports MAC operations

Clock

Synchronizes elements
State transitions on clock edge
(positive or negative)

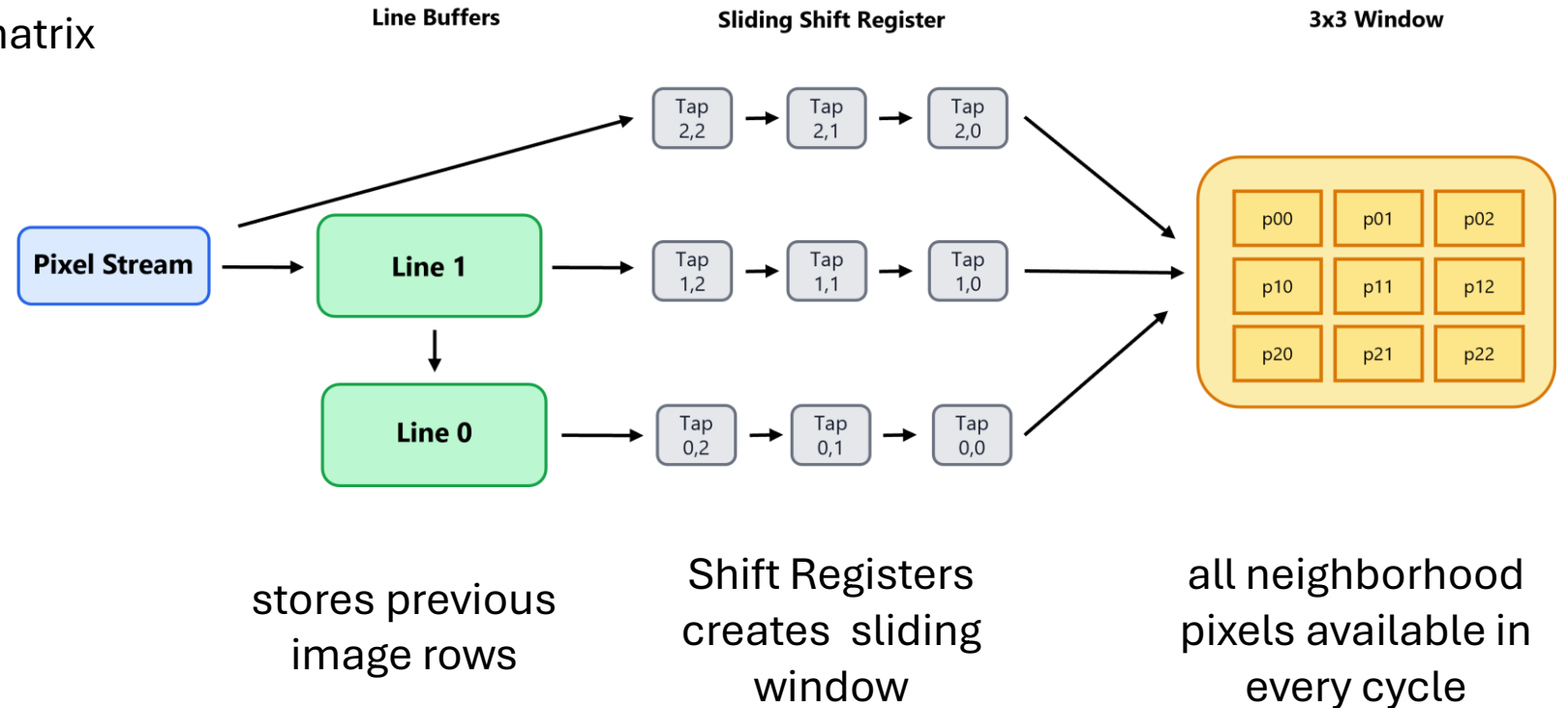


Line buffer (1)

- Efficient data access & storage mechanism
- Converts pixel stream to window matrix

For $K \times K$ window $\rightarrow$
 $K - 1$ full lines stored

Warm-up latency
 $L = (K - 1) \cdot \text{Width} + K$



Line buffer (2)

Problems

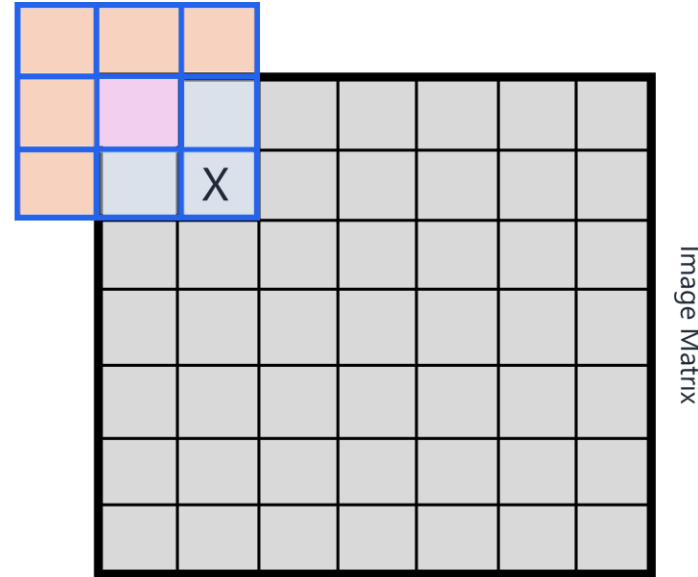
- ① pixels at the right edge of line y can blend with the left edge of line $y+1$
- ② window can be out-of-bounds



Coordinate tracker for the last input pixel

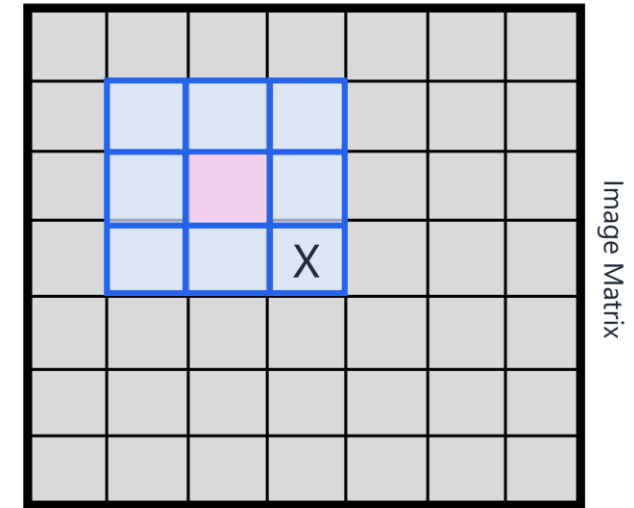
Valid window when:

$$W - 1 \geq \text{in_col} \geq 2 \text{ and } H - 1 \geq \text{in_row} \geq 2$$



Kernel out of bounds

X: $\text{in_col}, \text{in_row} = 1$



Kernel within bounds

X: $\text{in_col}, \text{in_row} \geq 2$

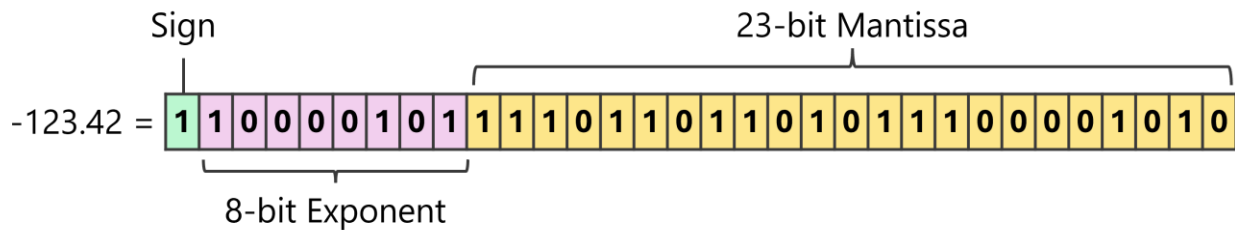
Image dimensions decrease by $\frac{K-1}{2}$

Floating-point arithmetic

IEEE 754 single-precision format

uses 32bits → 1-bit sign (S)
8-bit biased exponent (E)
23-bit mantissa or fractional (M)

expressed as: $A = (-1)^S \cdot (1.M) \cdot 2^{E-127}$



- Arithmetic operations work on the components (S, E, M)
- Normalized and reassembled result
- High resource usage (DSPs)

4 floating-point modules

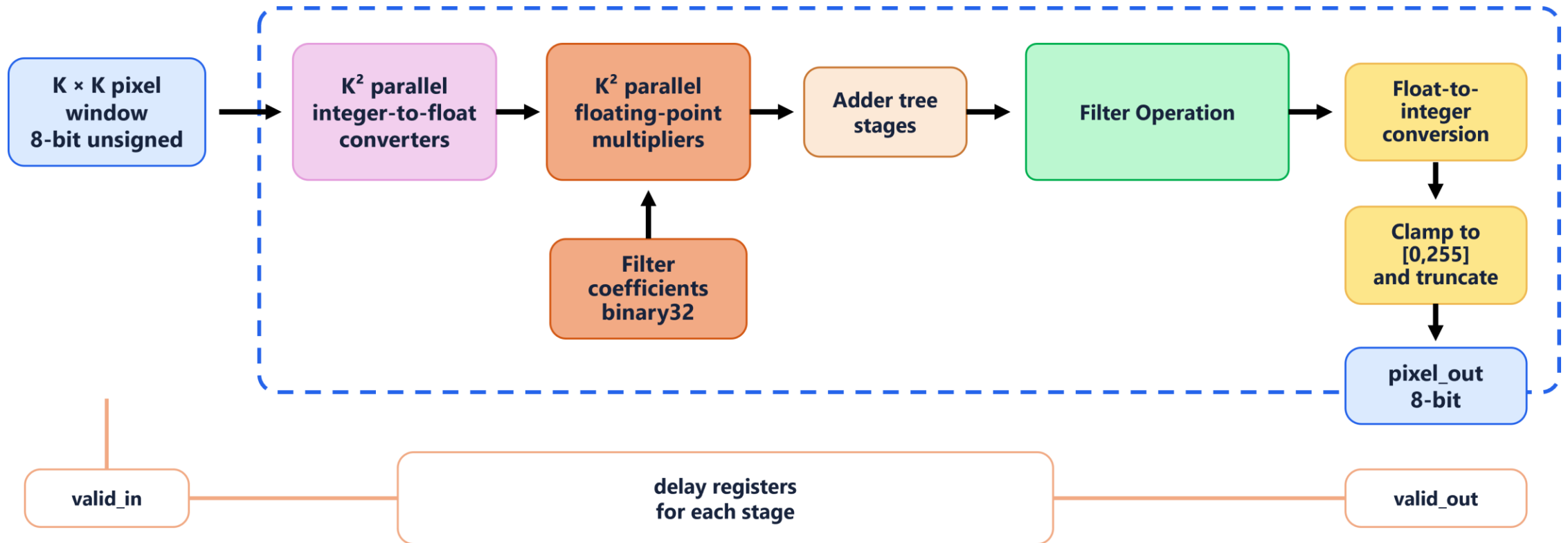
Integer-to-float
converter

Float adder

Float
multiplier

Float-to-integer
converter

Floating-point architecture



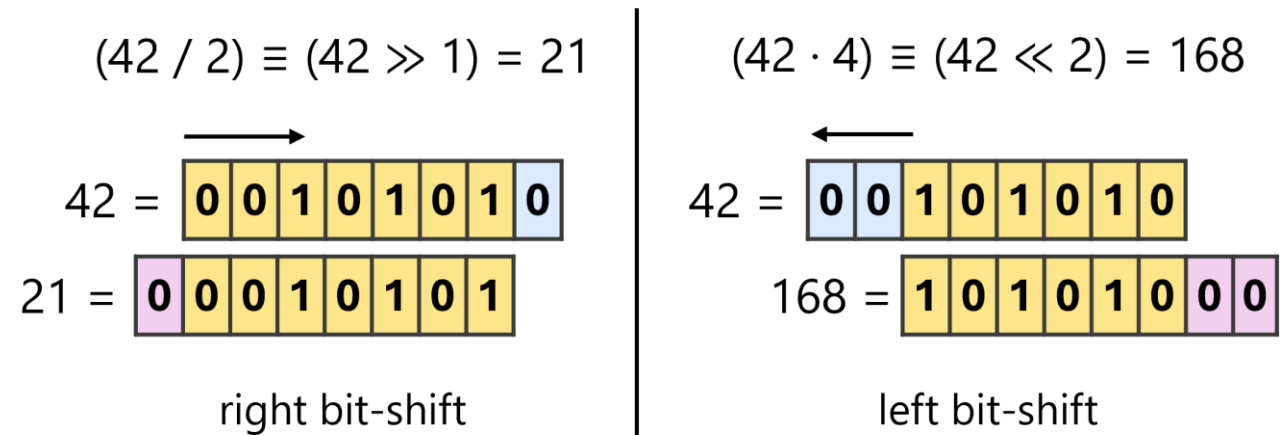
Fixed-point arithmetic

Integers = special case of fixed-point numbers where the fractional part is zero

- Multiplication & division using bit-shifts
- Results are summed through an adder tree
- Low implementation complexity

inexpensive calculations

no multipliers $\Rightarrow$ 0 DSPs



Example of bit-shift for multiplication and division

Sobel filter

- Edge detection algorithm
- Two directional kernels with integer coefficients
- Each kernel produces a partial gradient
- Exact final gradient: $G = \sqrt{G_x^2 + G_y^2}$
- Approximation used for simpler calculations

3x3 kernels

$$K_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

$$K_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

5x5 kernels

$$K_x = \begin{bmatrix} -1 & -2 & 0 & 2 & 1 \\ -4 & -8 & 0 & 8 & 4 \\ -6 & -12 & 0 & 12 & 6 \\ -4 & -8 & 0 & 8 & 4 \\ -1 & -2 & 0 & 2 & 1 \end{bmatrix}$$

$$K_y = \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 2 & 8 & 12 & 8 & 2 \\ 0 & 0 & 0 & 0 & 0 \\ -2 & -8 & -12 & -8 & -2 \\ -1 & -4 & -6 & -4 & -1 \end{bmatrix}$$

↓
 G_x

↓
 G_y

final gradient

$$G = |G_x| + |G_y|$$

Gaussian filter

- Blurring filter, smooths details, reduces noise
- Circularly symmetric Gaussian
- Each pixel has a different weight based on its distance from the center (x,y)

3x3 kernels

Integer approximation

$$K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Exact values

(0,0)	0.331910665
(0,1)	0.12210311
(1,1)	0.044919224

5x5 kernels

$$K = \frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$

(0,0)	0.162102822
(0,1)	0.098320328
(1,1)	0.059634294
(0,2)	0.021938231
(1,2)	0.01330621
(2,2)	0.002969017

Cascaded filters

two filters in sequence: Gaussian blur → Sobel filter

extra line buffer between the two cores

valid signal handshake chain across stages

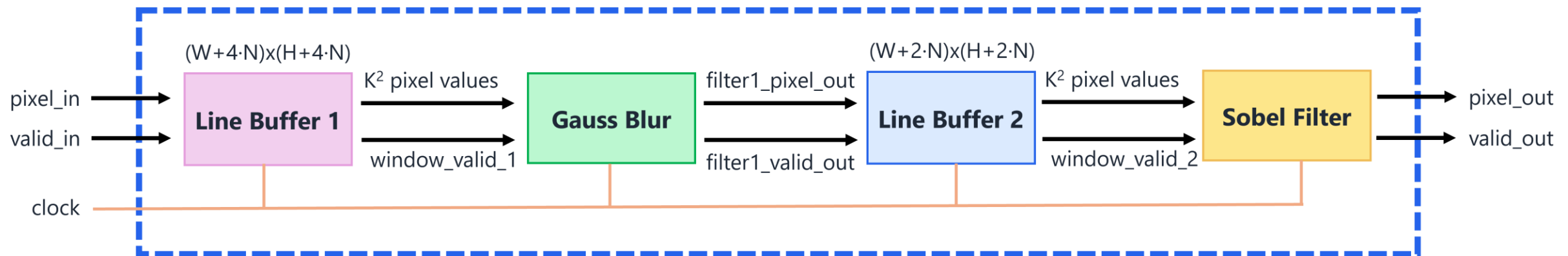
Integer cascade

Intermediate value is truncated to 8-bit
error accumulates

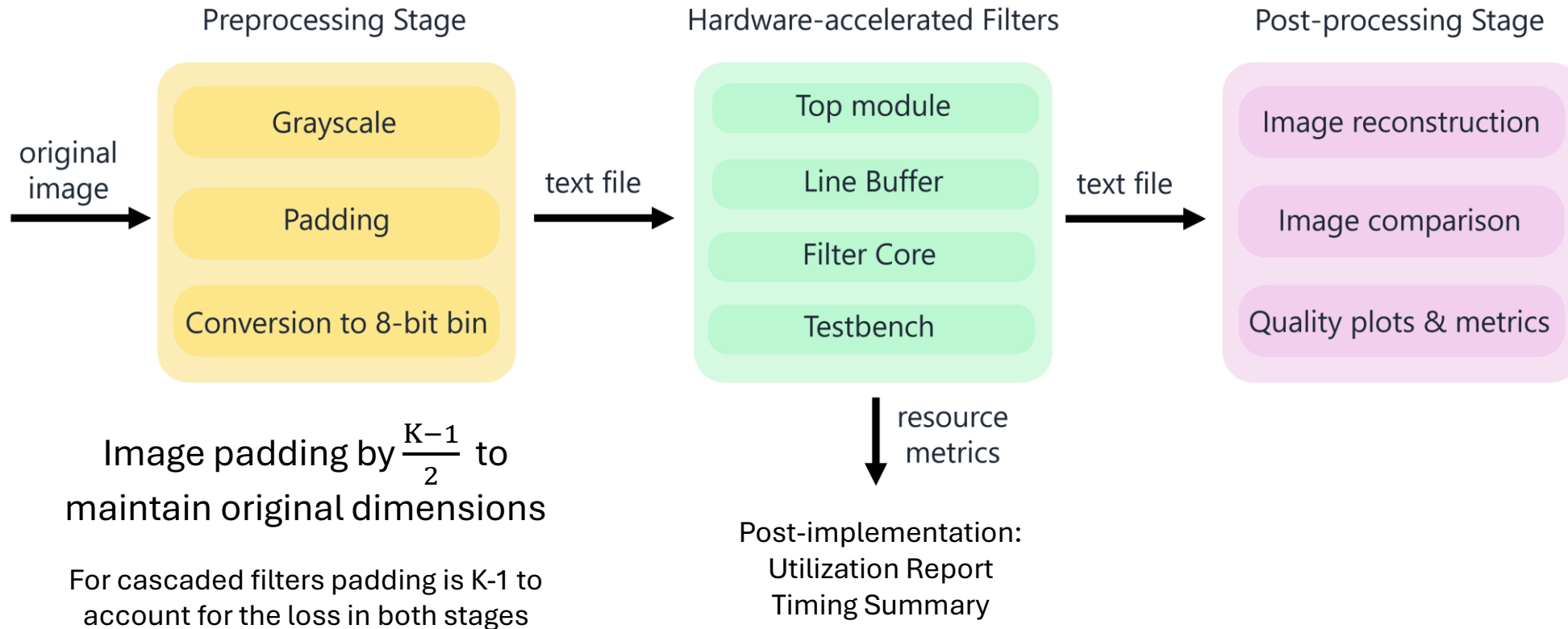
Floating-point cascade

Intermediate value is transferred as a float
no precision is lost between stages

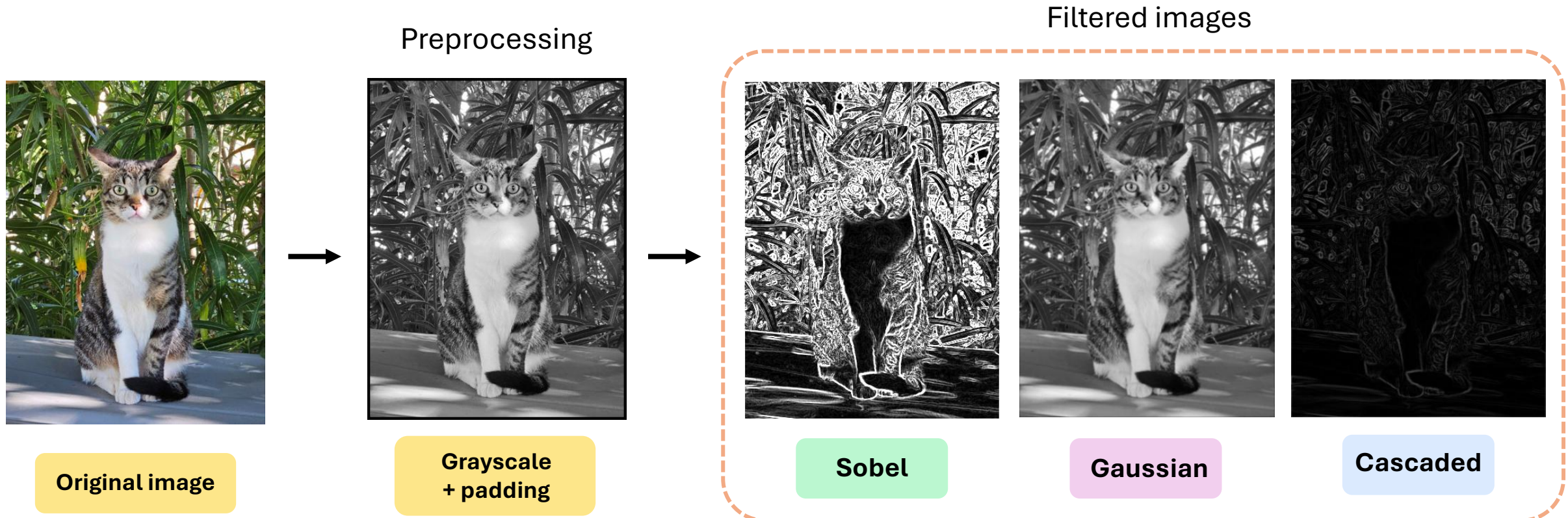
Top Module Wrapper – Cascaded filters



Workflow (1)



Workflow (2)



Research questions

- ① How does floating-point arithmetic affect image quality compared to integer fixed-point?
- ② How does hardware resource consumption change when the kernel size increases from 3x3 to 5x5?
- ③ How much do errors accumulate when connecting multiple filters in a sequence?
- ④ How does frame rate and area efficiency differ between implementations and kernel sizes?

Fixed

target device
test image
streaming framework
evaluation scripts

Changing

kernel size
filter type
arithmetic representation

Experimental setup

12 designs in
SystemVerilog

AMD Xilinx Vivado Design Suite 2025.2

Artix-7 FPGA (xc7a200tsbg484-1)

$$T_{\text{target}} = 20\text{ns} \Rightarrow F_{\text{target}} = 50\text{MHz}$$

Image quality metrics:

- Maximum Absolute Difference (**MAD**)
 - Root Mean Squared Error (**RMSE**)
 - Peak Signal-to-Noise Ratio (**PSNR**)
 - Structural Similarity Index Measure (**SSIM**)
-

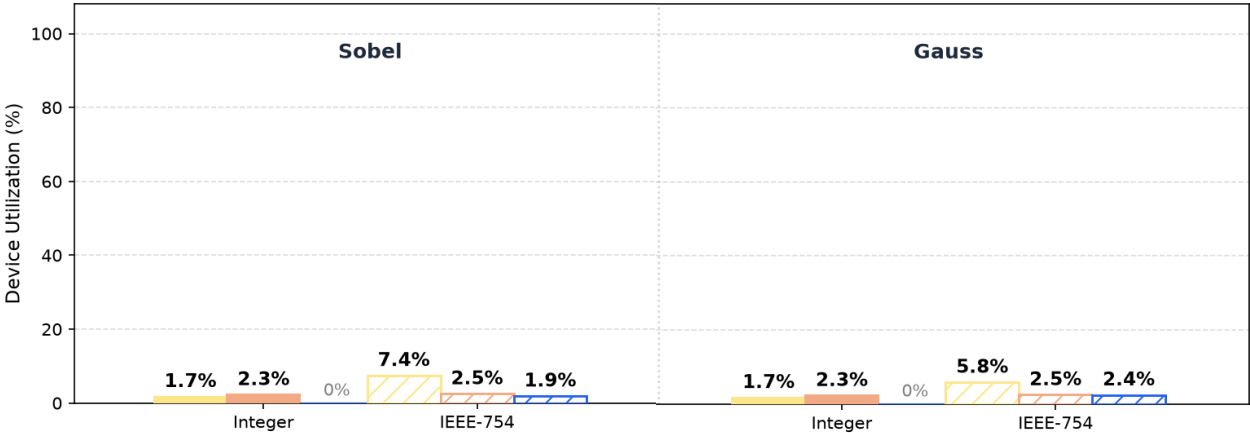
Hardware resource metrics:

- Look-Up Tables (**LUTs**)
- Flip-Flops (**FFs**)
- Digital Signal Processing Blocks (**DSPs**)
- Worst Negative Slack (**WNS**)
- Maximum operating frequency (**F_{max}**)
- Frame rate (**FPS**)
- Throughput (**T**)

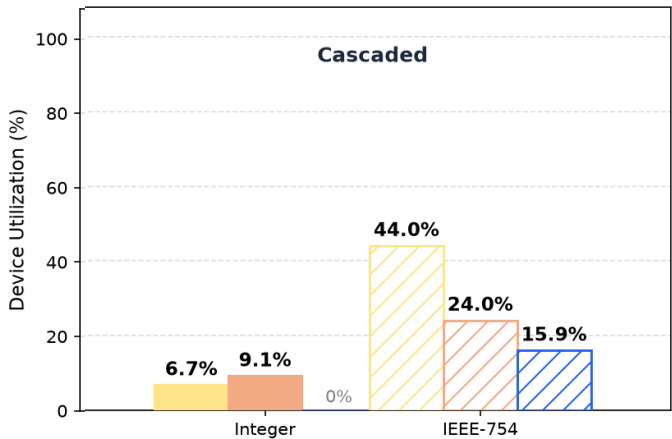
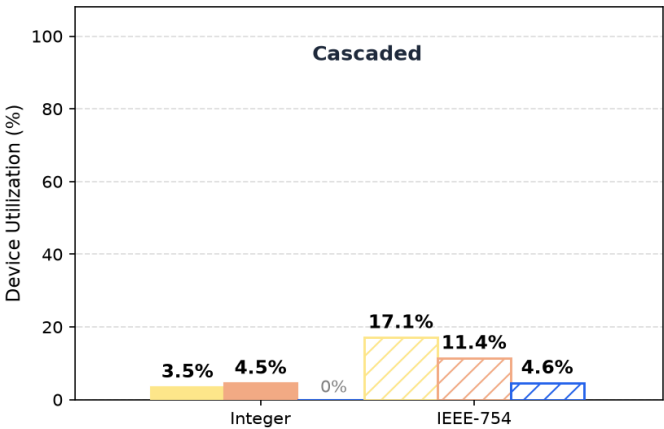
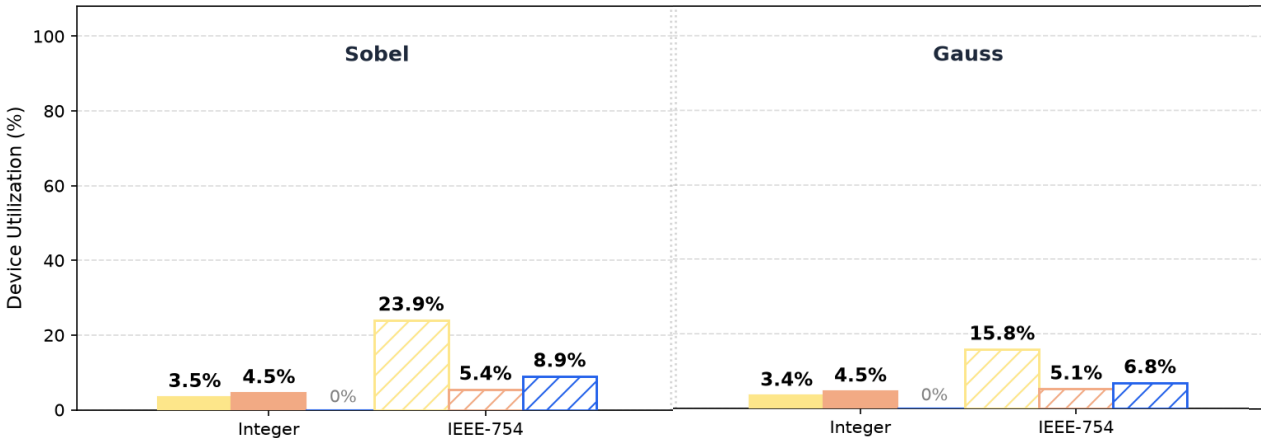
Resource metrics

Percentage of the total board resources used

3x3 Window



5x5 Window

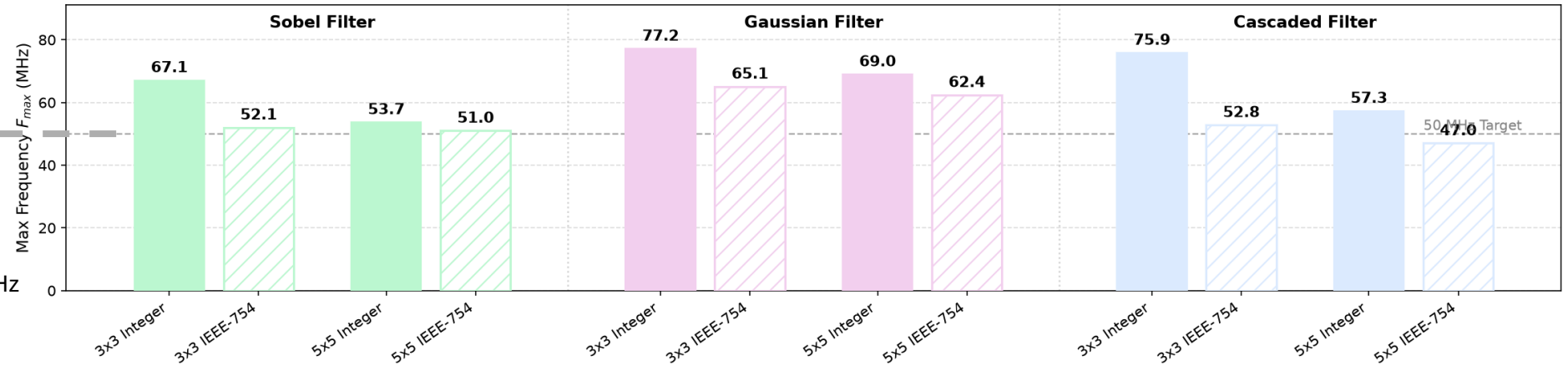


Timing

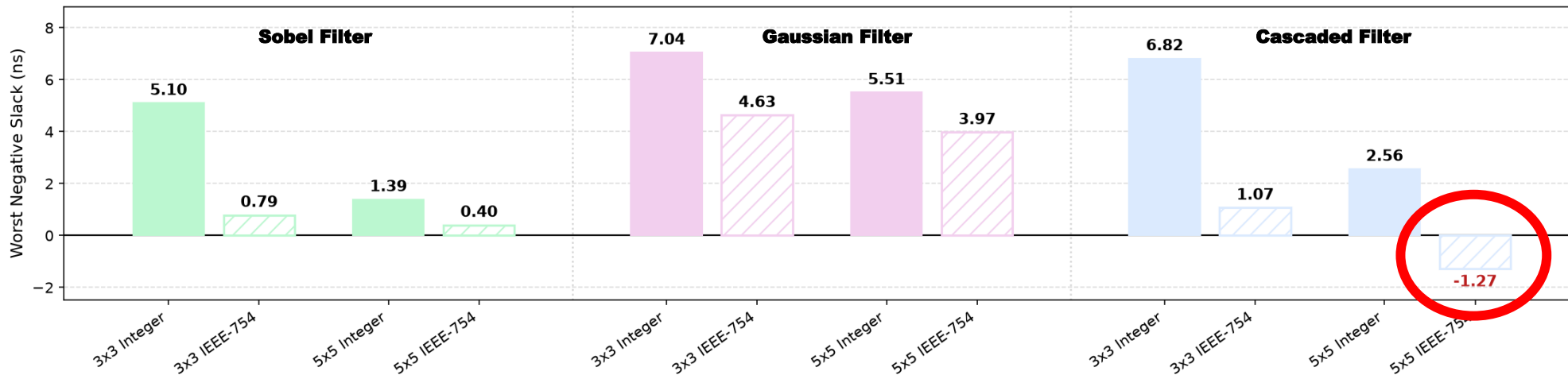
$$F_{\max} = \frac{1}{T_{\text{target}} - \text{WNS}}$$

$F_{\text{target}} = 50\text{MHz}$

Maximum operating Frequency ($F_{\max}$)



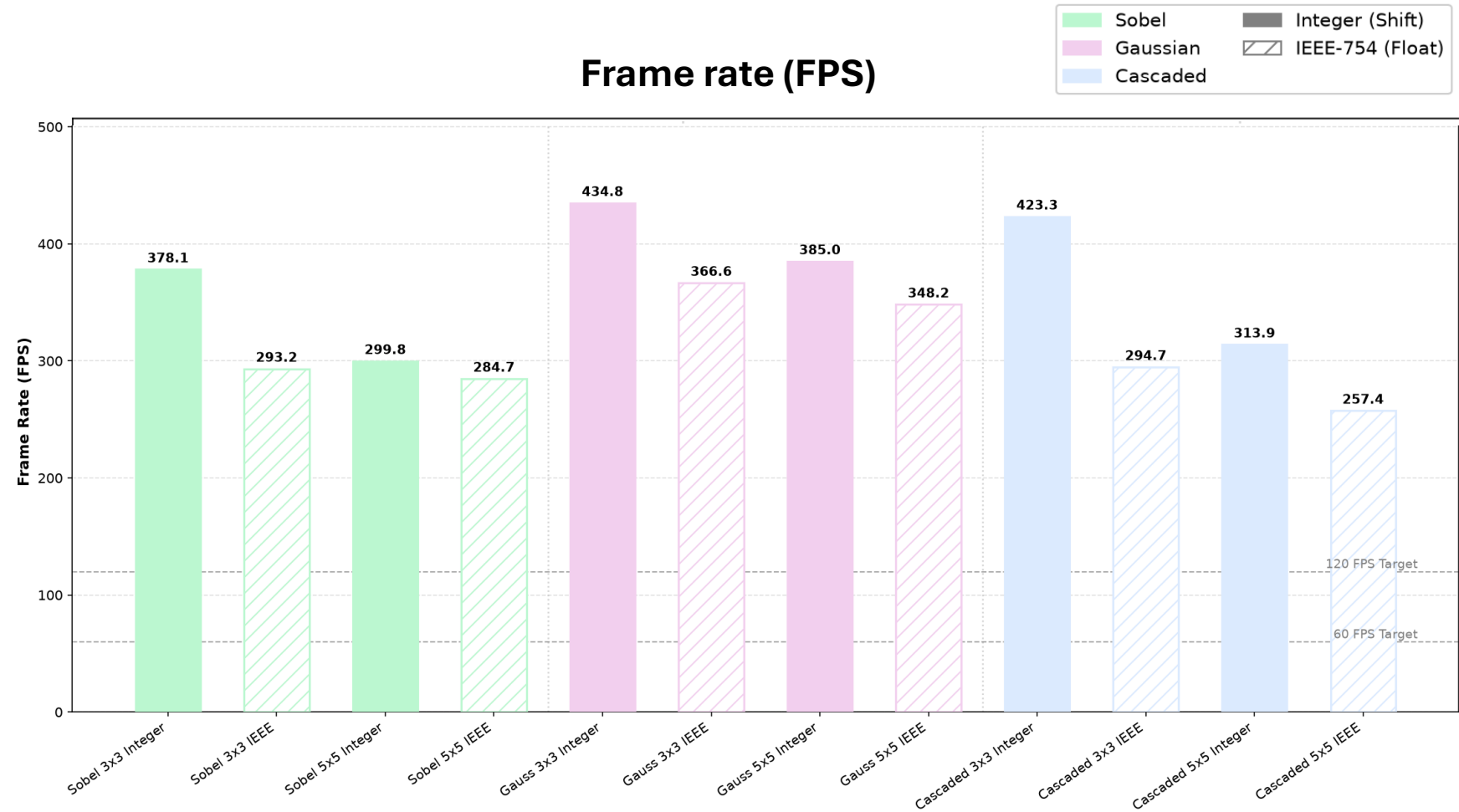
Worst Negative Slack (ns)



5x5 Cascade floating-point doesn't have timing closure

Frame rate

$$\text{FPS} = \frac{F_{\max}}{\text{Total Cycles per Frame}}$$



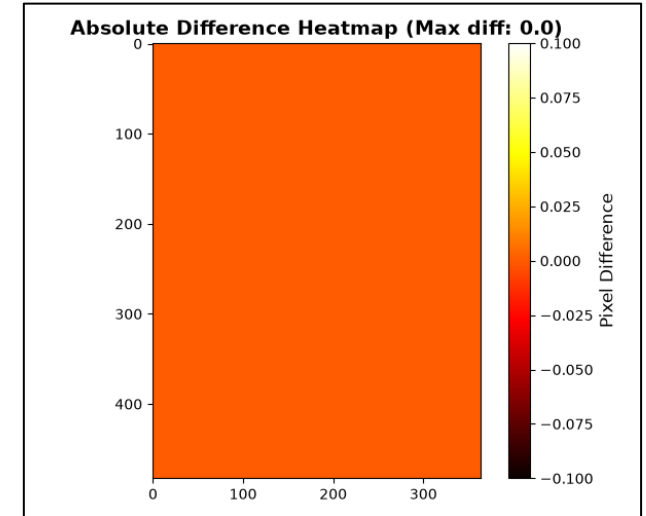
Results: Sobel

All Sobel filter cases had **100% similarity** with the reference

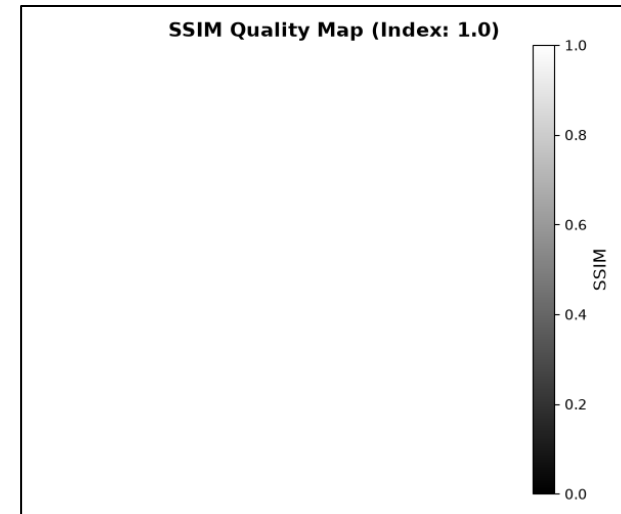
MAD	0
RMSE	0
SSIM	1

Integer kernel coefficients $\rightarrow$ No accuracy gained from floating-point design

Absolute difference heatmap



SSIM Quality Map (Index: 1.0)

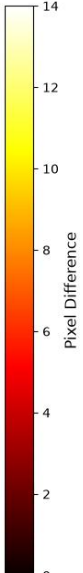
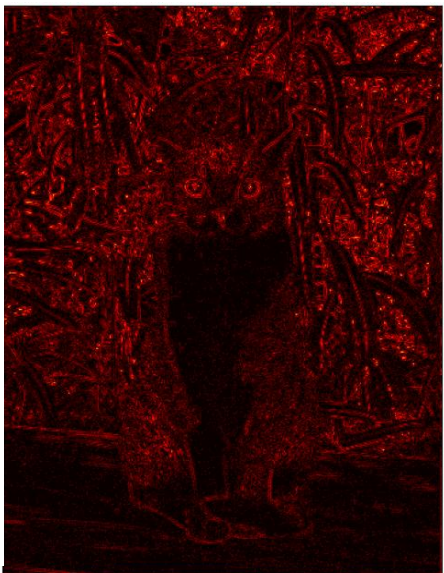


SSIM quality map

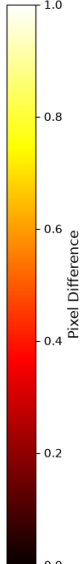
Results: Gaussian

3x3 window

Integer



Floating-point

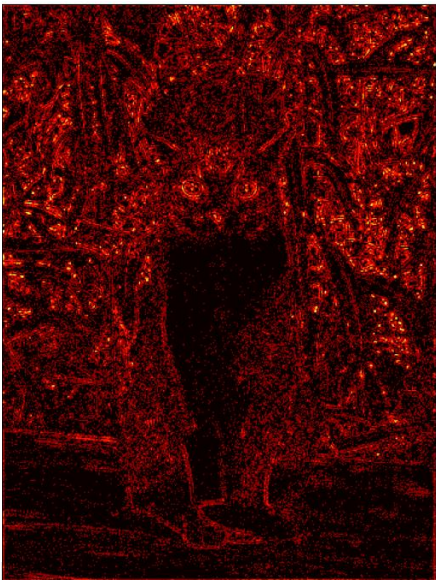


MAD	14
RMSE	1.396181
SSIM	0.99741

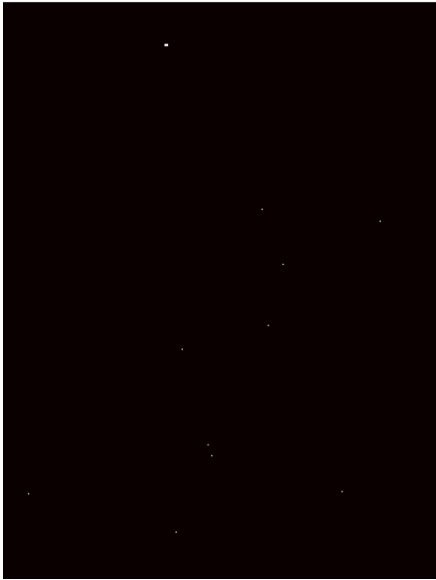
MAD	1
RMSE	0.086517
SSIM	0.999913

5x5 window

Integer



Floating-point



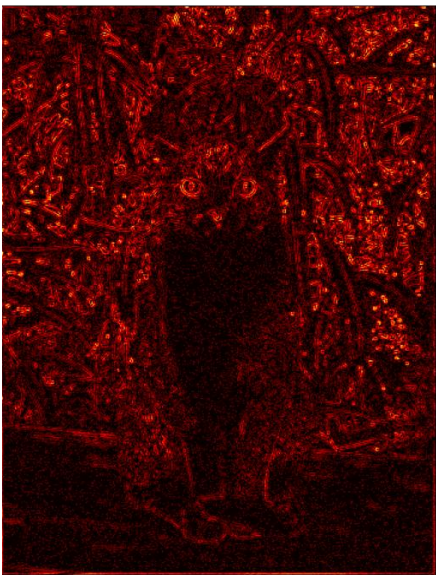
MAD	5
RMSE	0.752312
SSIM	0.998973

MAD	1
RMSE	0.009833
SSIM	0.999999

Results: Cascade

3x3 window

Integer



Floating-point

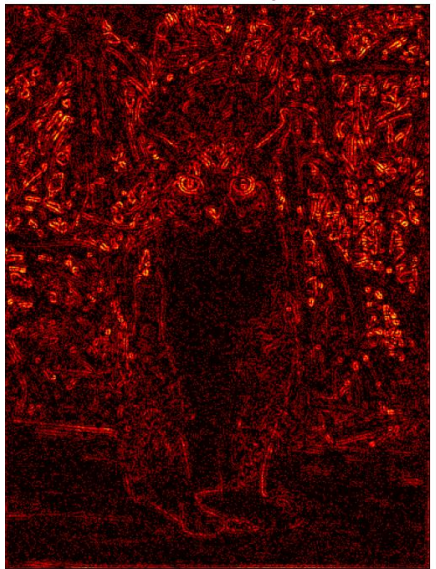


MAD	9
RMSE	1.194043
SSIM	0.994982

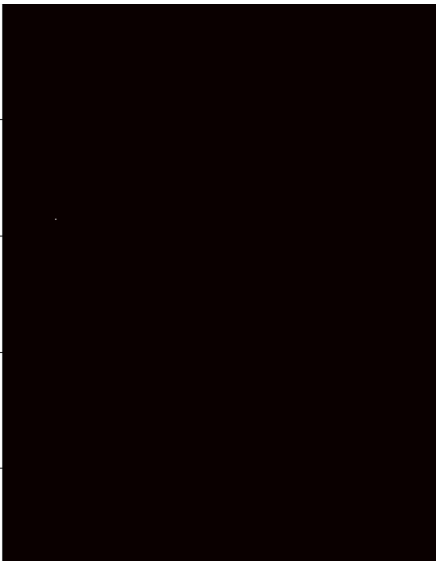
MAD	1
RMSE	0.00954
SSIM	0.999998

5x5 window

Integer



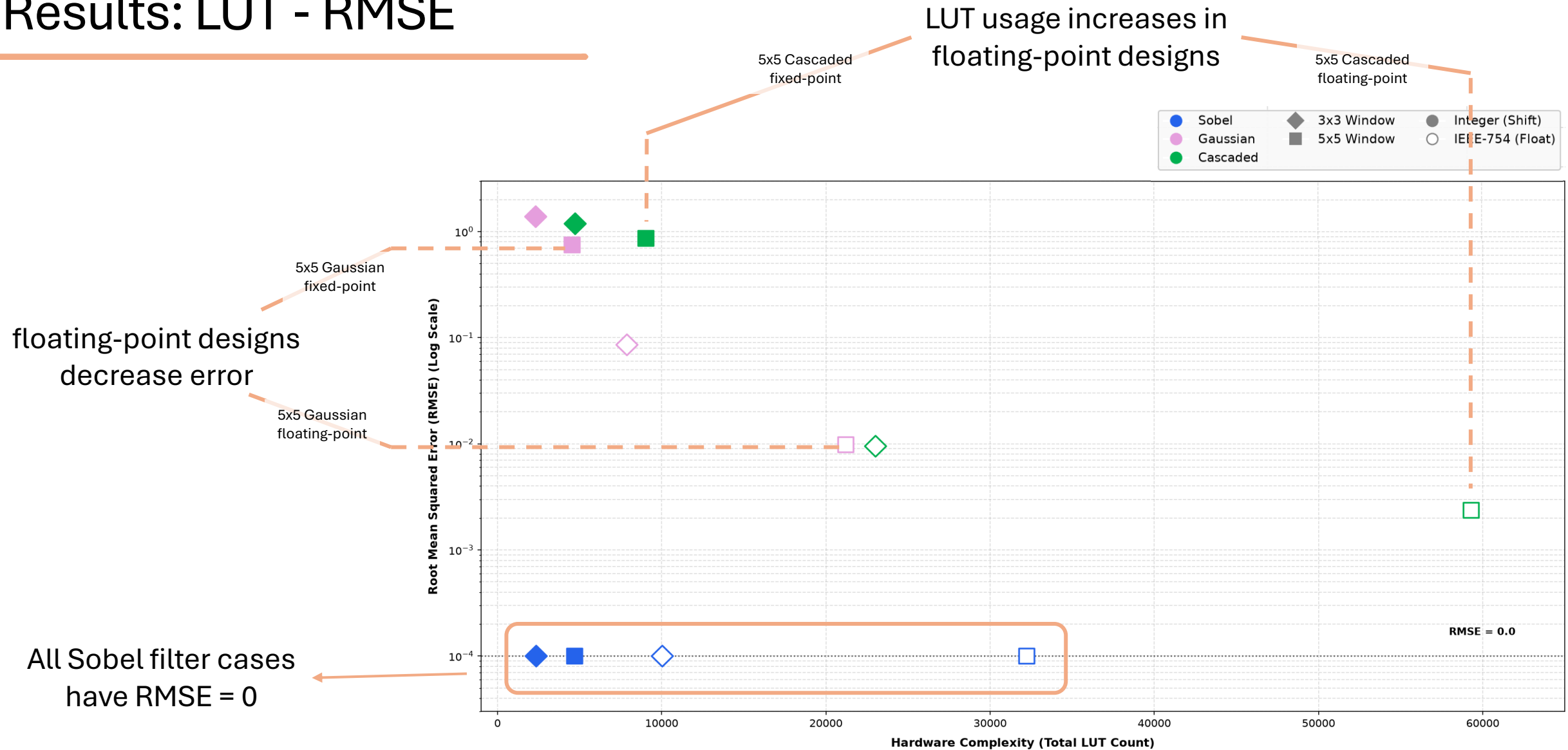
Floating-point



MAD	6
RMSE	0.870081
SSIM	0.998019

MAD	1
RMSE	0.002385
SSIM	0.999999

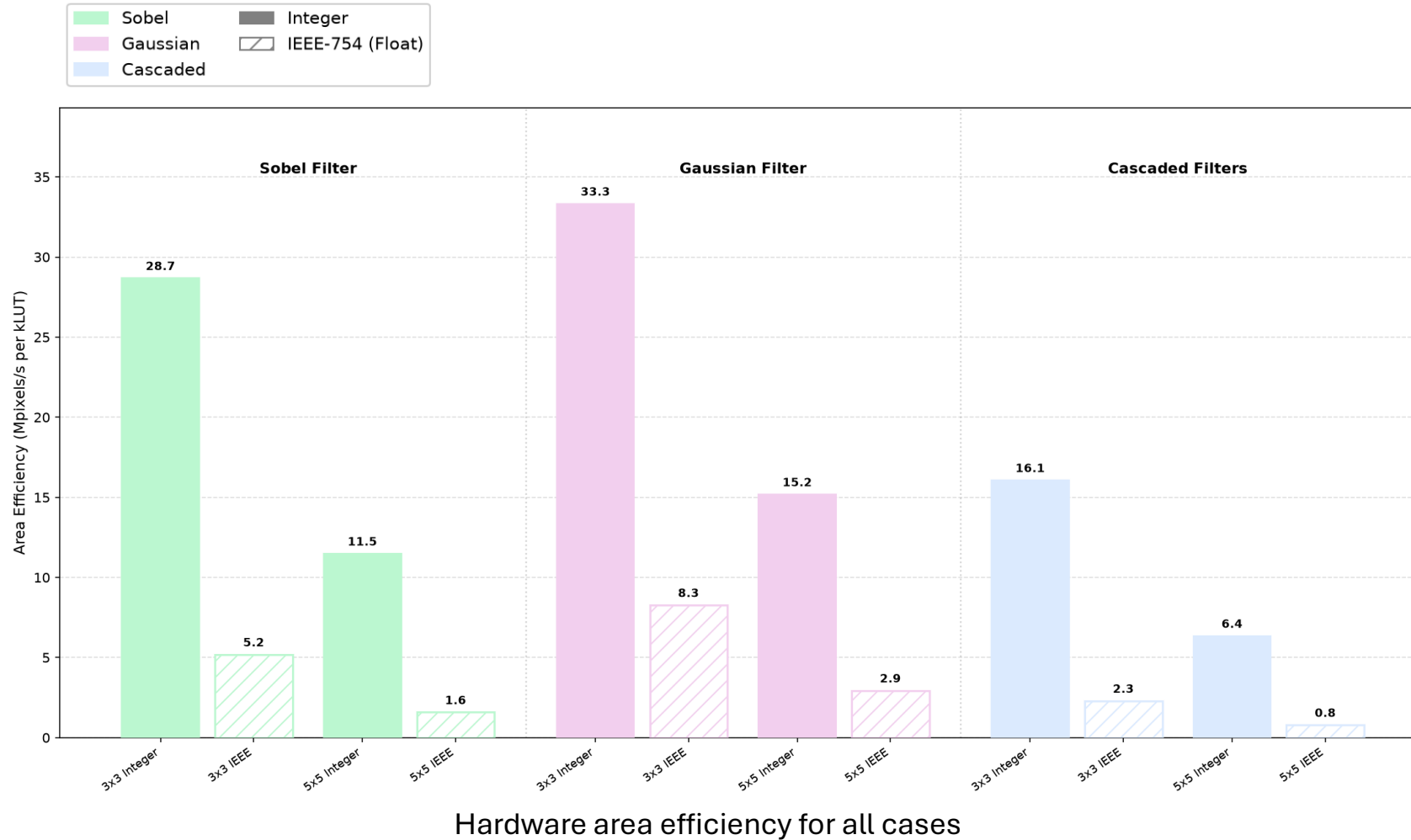
Results: LUT - RMSE



All Sobel filter cases
have RMSE = 0

Total number of LUTs and the achieved RMSE

Results: Efficiency



Throughput

how many pixels
can be processed
per second

Area Efficiency

Throughput per
1000 LUTs

Conclusion

- ① Floating-point arithmetic provides accuracy depending on the value of the coefficients

- ② Increase of the kernel size raises every hardware resource metric

- ③ Errors accumulate when connecting filters in a sequence (truncation in the integer design)

- ④ Frame rate and area efficiency decrease with kernel scaling and design complexity

Thank you!

Code & results
available at [Github](#).

